



**University of
Nottingham**
UK | CHINA | MALAYSIA

Faculty of Science and Engineering

Department of Computer Science

Computer Vision

Report : Panoramic Photo Generation

Course Code: COMP 3065

Student's Name : Nicholas LEE

Student's ID : 20413967

1 Introduction

Panoramic photography focuses on capturing an overview of a landscape due to the camera's limited field of view, which cannot capture an all-around view over the entire horizon. This report shows how to create a cylindrical panoramic image using an all-around video captured by a standard digital camera without wide-range capability. The outcomes of the experiment are discussed, along with the strengths and weaknesses of the overall program.

1.1 Objectives

The primary goals of this work are to:

- Build a complete panoramic photo generator that converts ordinary 360-degrees video into panoramic images.
- Deliver a user-friendly desktop application that enables non-experts to generate panoramas from video files with minimal effort.

1.2 Key Features

The implementation includes:

- **Graphical User Interface (GUI):** A Tkinter-based UI (Figures 1 and 2) for selecting the input video and choosing an output directory.
- **Automated Frame Extraction and Alignment:** Intelligent sampling of representative frames combined with feature detection (SIFT) and RANSAC homography to ensure accurate geometric alignment.
- **Smooth Stitching and Cleanup:** Integration with OpenCV's Stitcher for blending, followed by mask-based cropping to produce a clean, artifact-free panorama.

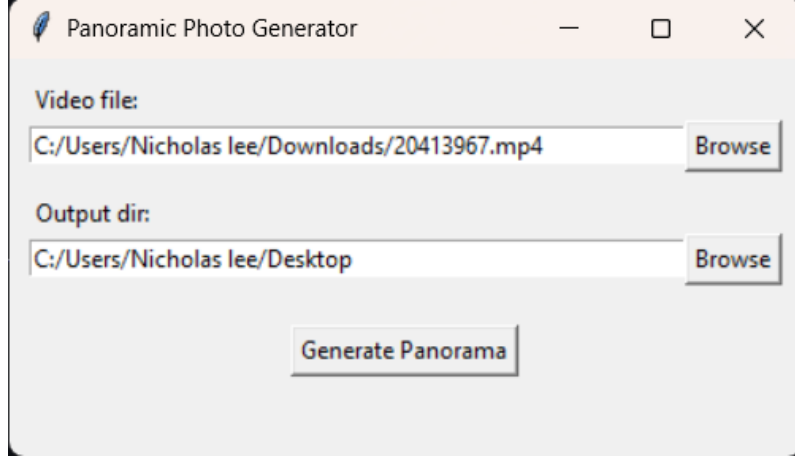


Figure 1: Panoramic Photo Generator GUI

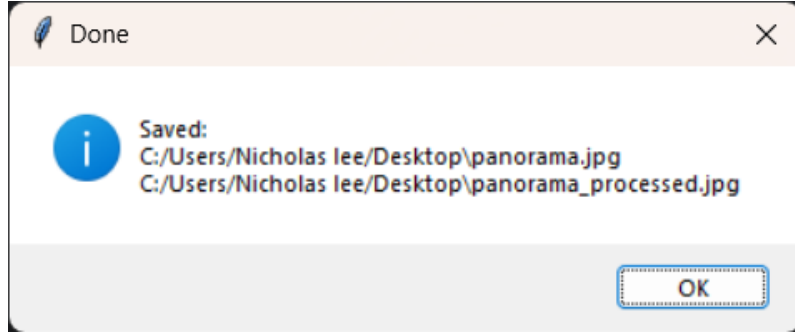


Figure 2: Panoramic Photo Generator GUI

2 Methodology

2.1 Frame Extraction

Panoramic preprocessing begins by loading the video stream with `cv2.VideoCapture`, enabling frame-by-frame access. Because subsequent frames share significant visual overlap, keeping the full sequence would add little information gain while greatly increasing computation resources. To reduce redundancy, the algorithm keeps only those frames whose index is an exact multiple of a user-specified stride (default = 40) and logs each acceptance for traceability. However, to ensure efficiency, the stride must be adjusted for each video.

After this sampling, each candidate frame is compared with the most recently accepted key frame. The comparison counts feature descriptors that the two frames have in common. If the match count exceeds an upper limit, the frame is considered

too similar and is removed; if it falls below a lower limit, the frame lacks sufficient overlap and is also discarded. So, retained frames cover about two-thirds of the previous key frame. This gives reliable feature mapping enough shared material without having to waste time on nearly identical frames. The final set of key frames covers the whole scene, makes matching more reliable, and greatly reduces the amount of work that needs to be done on the computer during picture stitching.

2.2 Feature Detection and Description

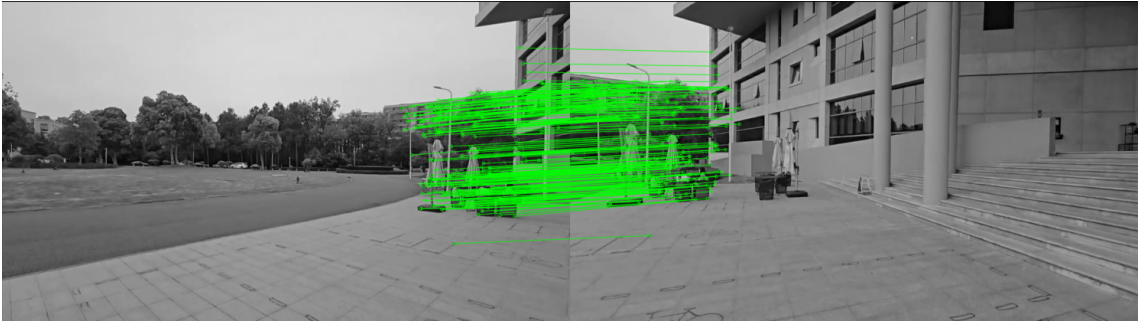


Figure 3: SIFT Example

After preprocessing, each frame is converted to grayscale and processed with Scale Invariant Feature Transform (SIFT) to detect distinctive keypoints and compute their 128-dimensional descriptors [1]. Converting to grayscale removes colour noise and speeds gradient calculation, while SIFT’s scale-space detector keeps points stable under zoom and rotation. A brute-force matcher with L_2 distance performs k -nearest-neighbour matching ($k = 2$), and Lowe’s ratio test (threshold = 0.6) filters out ambiguous correspondences, retaining only the most reliable third of tentative matches so geometry later sees clean data. Frame pairs with fewer than four surviving matches are skipped to avoid unstable homography fits, while the rest feed into a Random Sample Consensus (RANSAC) routine (reprojection threshold = 5 px) that estimates a strong homography and removes outliers [2]. The resulting per-pair homographies capture the planar alignment between consecutive frames, chaining them cumulatively ($H_1, H_2H_1, \dots$) keeps memory flat and supplies a stable geometric map for downstream blending and exposure correction in the panorama pipeline. Figure 3 illustrates key points’ matching using SIFT.

2.3 Frame Stitching

Frame stitching is handled by the `stitch_images(images)` function, which is an OpenCV's API [3]. A `cv2.Stitcher` instance is created via `cv2.Stitcher_create()`, and the ordered list of key frames is passed to `stitcher.stitch(images)`. The returned `status` code is compared against `cv2.STITCHER_OK`; if they match, the assembled panorama (`pano`) is returned. Otherwise, an error message is printed and `None` is returned, allowing the program to detect and report failures.

Beyond this basic logic, the Stitcher engine matches pairwise, first extracts scale-invariant features (the default is ORB [4] or AKAZE [5], depending on OpenCV build parameters), and computes a set of candidate homographies via a whole pipeline internally. Following which photos are distorted into a common projection (cylindrical or spherical by default), a bundle-adjustment step then refines camera positions globally to avoid reprojection error. Following graph-cut seam discovery that chooses the best transition lines where overlap is significant and texture variation is minimal, exposure correction is applied per-camera to equalize gain and color cast. Ultimately, multi-band (Laplacian pyramid) mixing smooths those seams across frequency ranges, hence reducing visible edges even in cases of varying illumination between frames. Figure 4 presents some examples of the frame stitching results.



Figure 4: Frame Stitching Pre-processing Examples

2.4 Post-processing

Post-processing begins by keeping valid picture content at the boundaries during masking, the stitched panorama is first padded with a fixed-size border (default 10 px, black) around all sides using OpenCV's `copyMakeBorder` function. The bordered image is converted to grayscale and thresholded to form a binary mask. Non-black pixels (foreground) are set to 255.

After that, contours are recovered from the binary mask using `cv2.findContours`, and the biggest contour by area is found. Created onto an empty mask, the bounding rectangle is eroded iteratively, reducing it one pixel at a time and deleting from the content mask until valid content is removed. This results in a minimal mask that includes all panorama data. The original bordered image is trimmed to the bounding box of the eroded mask and written to disk using `cv2.imwrite`. Then, a light sharpening pass gives the panorama a neat, completed look by restoring crisp edges that might seem a little soft during stitching. (see Figure 5). This results in a tight, artifact-free panoramic output.



Figure 5: Sharpened Images Detail Examples

3 Results and Discussion

3.1 Results

Figure 6 shows UNNC outdoor campus scene in the library area, including the lawn and trees in the center and the building facade with steps on the right and left. Moving people are not ghosted, seams are clean and there is no obvious misregistration around stair rails or lamp posts. From grass to the sky, exposure and color are constant, the horizon is flat and straight lines exhibit no distortion. With the black border totally eliminated and the cropping is tight.

Figure 7 captures the 3rd canteen. The picture consists of seating areas, shelving units, and ceiling lights all aligned well. Although there is a small offset in the tile lines close to the far left pillar, general alignment is good. Colors and brightness remain constant, where there is no exposure change between ceiling lights and window light. Though a few pixels would contain more ceiling, the crop removes all black borders and centers content. Geometric lines remain straight, while the darker corners show just slight noise.

Figure 8 presents the interior panorama of the Portland Building, featuring the glass facade with evenly spaced windows, clusters of tables and chairs, a potted plant at the left, and the central digital display panel. Architectural details including window frames, railings, and ceiling beams show no mis-registration or distortion, seams are invisible and persons remain appropriately shown without ghosting. Exposure and color balance stay uniform from the tiled floor through to the upper windows, with no abrupt shifts between interior lighting and daylight. Only a minor artifact is seen next the left planter, which does not compromise the general image quality. Furthermore, the cropping has totally eliminated any black border, creating a tight, focused frame.



Figure 6: Panoramic result 1



Figure 7: Panoramic result 2



Figure 8: Panoramic result 3

3.2 Discussion

Strengths

1. **Efficient frame extraction, fast runtime.** Keeping only every n -th frame and removing frames with too little or too many overlap cuts memory and processing time without losing too much scene coverage.
2. **Reliable alignment.** SIFT + Lowe’s ratio test and RANSAC drop bad matches before the homography is estimated, so straight lines have no visible seams or ghosting (see Figures 6 and 7).
3. **Automatic blending and exposure balance.** OpenCV’s `Stitcher` performs exposure compensation and multi-band blending. For examples, indoor lamps and daylight windows look even and outdoor shots keep sky-to-grass colour steady.
4. **Clean GUI for amateur users.** The interface hides parameters behind file pickers, so a user can create a panorama with only a few clicks.

Weaknesses

1. **Too aggressive cropping.** The mask-erosion step that removes the black border sometimes erodes real content. In Figure 4 the right-hand railing is shaved off, and in Figure 8 parts of the glass facade are lost. In extreme cases, the panoramic photo can be cropped to a little stretch.
2. **Parameter sensitivity.** A stride that is too large leaves gaps and forces the stitcher to extrapolate, whereas a stride that is too small bloats the frame set and slows SIFT.
3. **Exponential computing cost on long 4K videos.** SIFT with brute-force matching scales poorly. Where a multi-minute 4K clip lags unless the stride is increased, which then reduces coverage.

3.3 Evaluation and Justification

- Visual quality: All three sample panoramas show perfect seams, flat horizons, and even color, proving that alignment and blending work.

- Failure cases: Cropping is the only source of serious artifacts. Whilst, alignment errors are minor and localized.
- Design Justification: Stride = 40 was chosen to balance 75% overlap (for robust homographies) against computational cost and a 10 px border ensures reliable mask margins without excessive erosion iterations.

4 Conclusion

The project was successful in accomplishing its objectives, which included transforming a standard 360-degree video into a clean panoramic image and doing so in a straightforward desktop graphical user interface that anyone can use with only a few clicks. The use of frame sampling helps to keep runtimes short, SIFT + RANSAC helps to align everything, and OpenCV’s Stitcher ensures that seams are eliminated by blending exposures. According to the results, the horizons are straight, the light is even, and there is nearly no ghosting.

References

- [1] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [2] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [3] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. Sebastopol, CA: O’Reilly Media, 2008.
- [4] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, “ORB: An efficient alternative to SIFT or SURF,” in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, (Barcelona, Spain), pp. 2564–2571, Nov. 2011.
- [5] P. F. Alcantarilla, A. Bartoli, and A. J. Davison, “Accelerated-KAZE features,” in *Proc. Eur. Conf. Computer Vision (ECCV)*, (Florence, Italy), pp. 825–838, Oct. 2012.